

**Leakage Audit of the PhiUSIIL Phishing URL Benchmark: Data Exploration,
Preprocessing, and Initial Model Development**

Group 5

J. Eric Collier, Alberto Kabore, Divya Kamepalli, Zubair Ahmed Baig Mogal,
and Bharadwaj Reddy Minukuri

School of Computer and Information Sciences, University of the Cumberland

PhDAI 732-A03: Fundamentals of AI-Enabled Systems

Dr. Manjunath Paramashivaiah

September 20, 2026

Leakage Audit of the PhiUSIIL Phishing URL Benchmark: Data Exploration, Preprocessing, and Initial Model Development

Introduction and Problem Statement

Phishing remains a primary initial-access vector, and URL-level classification is attractive because it can act before a page is fetched. The PhiUSIIL Phishing URL Dataset is a widely used benchmark for that task, containing 235,795 labeled URLs described by 54 features spanning URL structure, domain properties, and rendered page content (Prasad & Chandra, 2024). Published classifiers routinely report accuracy above .99 on it, and URL-only approaches in the broader literature report comparable figures (Sahingoz et al., 2019). Our first run reproduced the benchmark result exactly. A random forest fit on the complete feature set scored 1.0000 on a held-out partition of 58,843 records.

A perfect score on a security problem is not a finding. It is a signal that the model holds information it could not possess at the moment a prediction is made. Kaufman et al. (2012) define that condition as leakage, and Kapoor and Narayanan (2023) document its consequences across 294 papers in 17 scientific fields, where corrected analyses frequently erase the reported advantage of machine learning over simpler methods. The risk is not that a leaking model fails a test. It is that the model passes, gets cited, and fails only in deployment.

Group 5 therefore designed Part 1 as a leakage audit rather than a classification contest. Two stakeholder groups depend on the answer. Security practitioners who would deploy a classifier trained on this benchmark need to know whether its reported accuracy transfers to live traffic. Researchers who cite PhiUSIIL results need to know what those results measure. The question this deliverable answers is not how accurately the classes can be separated, but what the separation is made of.

Dataset Exploration and Preprocessing

Preprocessing was implemented in pandas and executed once, with every output written to disk as JSON or CSV before any section cited it. The raw file contained 235,795 observations across 56 columns. Screening identified 425 duplicate URLs and no fully duplicated rows, and

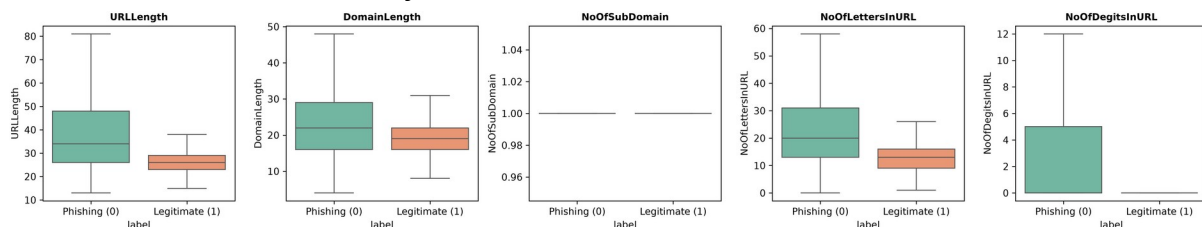
no URL carried conflicting labels. Removing the duplicates left 235,370 observations with no missing values across 220,086 unique domains. Label 1 denotes a legitimate URL and holds 134,850 records, or 57.29% of the cleaned data; label 0 denotes phishing and holds 100,520 records, or 42.71%. Because five members downloaded the dataset independently, a SHA-256 fingerprint of the sorted URL column was recorded and verified on each machine, which guards against silently comparing metrics computed on different bytes.

Exploratory analysis was implemented as a separate module that reads the data steward's frozen outputs rather than recomputing them, which keeps the published figures and the reported statistics from diverging. Class distribution poses no modeling difficulty. At 134,850 legitimate records against 100,520 phishing records neither class is rare, and the majority-class floor that any model must beat is .5729 accuracy.

The lexical distributions are less ordinary (Figure 1). Phishing URLs are longer at the median, 34 characters against 26, but dispersion separates the two classes more sharply than location does. The legitimate interquartile range spans 6 characters against 22 for phishing, so legitimate URLs are not merely shorter but far more uniform. Digits show the same asymmetry in a starker form: legitimate URLs average .051 digits with an interquartile range of zero, while phishing URLs average 4.324. One lexical feature carries almost no class signal at all. NoOfSubDomain has an interquartile range of zero in both classes and class means of 1.162 and 1.169, which leaves it contributing essentially nothing to the feature set it belongs to. Uniformity concentrated in one class, rather than a difference in central tendency, is the pattern the rest of this section pursues.

Figure 1

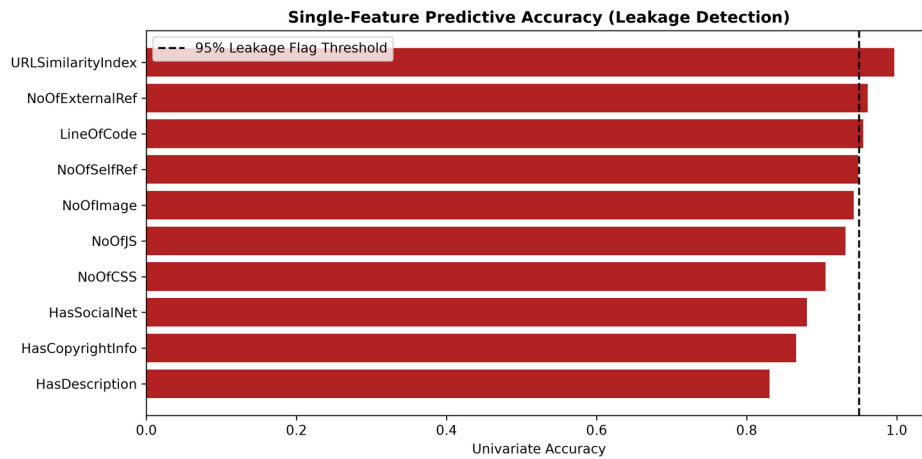
Lexical Feature Distributions by Class



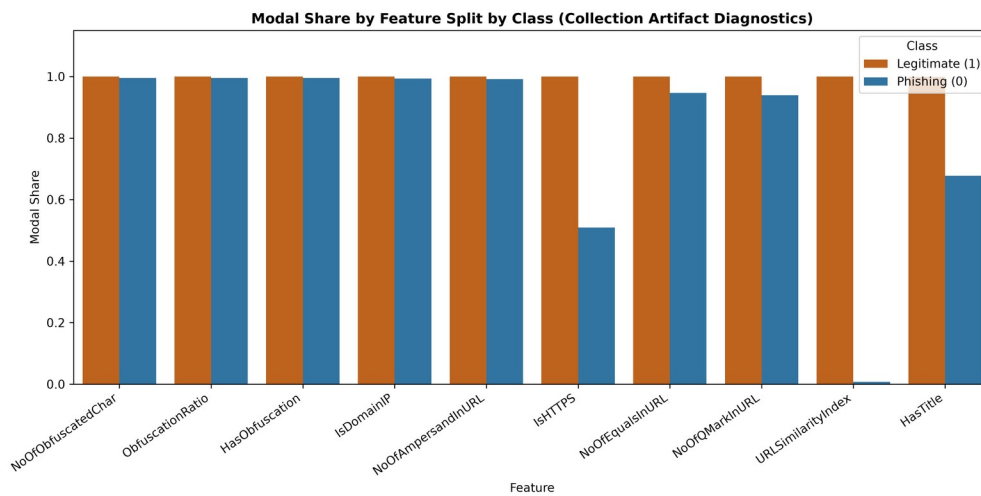
Note. Boxes span the interquartile range with the median marked; outliers are suppressed. Legitimate URLs are more tightly distributed than phishing URLs on every measure shown. NoOfSubDomain separates the classes on neither location nor spread. Quartiles and class means are recorded in results/lexical_distribution_summary.csv.

A single-feature screen then fit a classifier on each column in isolation. URLSimilarityIndex reaches .9967 accuracy alone (Figure 2). Its value is exactly 100 for all 134,850 legitimate records, with zero variance. The feature does not describe a URL. It restates the label, which follows directly from its construction as a similarity measure taken against a list of known-legitimate URLs. Three page-content features also clear .94: NoOfExternalRef at .9613, LineOfCode at .9554, and NoOfSelfRef at .9485. These describe a rendered page, which means they are unavailable at the moment a URL must be judged, before anything has been fetched. A model that uses them answers a different question than the one phishing detection poses.

Modal-share analysis located a second and more consequential problem. Eight lexical features hold a single constant value across the entire legitimate class, and URLSimilarityIndex makes nine (Figure 3). Every legitimate URL is served over HTTPS. None contains an ampersand, an equals sign, or a question mark, which means none carries a query string. None uses a bare IP address in place of a domain, and none contains an obfuscated character. By contrast, no feature is perfectly constant within the phishing class, where the highest modal share is .9957. Genuine web traffic does not distribute this way. The pattern is consistent with a legitimate class harvested by an HTTPS-only crawl of bare homepages and a phishing class drawn from a feed subject to no comparable filter.

Figure 2*Single-Feature Accuracy for the Ten Most Separating Columns*

Note. Each bar is the accuracy of a classifier fit on that column alone. The dashed line marks the .95 flag threshold. URLSimilarityIndex is construction-derived; NoOfExternalRef and LineOfCode require the page to be fetched and rendered.

Figure 3*Modal Value Share by Class for the Most Constant Features*

Note. A modal share of 1.0 means every record in that class carries the same value. Nine features are perfectly constant within the legitimate class. None is perfectly constant within the phishing class.

No individual lexical feature reaches the .95 threshold that flags leakage under a single-feature screen, so the standard audit fires on none of them. The two classes nevertheless differ systematically in shape before any single feature is examined. This is leakage in the sampling frame rather than in a column, and it is not repairable by feature ablation.

Three nested feature sets were defined once in a shared configuration module and frozen: full (all 50 modeling features), no_derived (44 features, removing the six construction-derived variables including URLSimilarityIndex), and url_only (the 18 lexical features readable from the URL string before any page is fetched). A stratified 75/25 split with seed 42 produced 176,527 training and 58,843 test records. A domain-grouped split was frozen alongside it, producing 176,906 training and 58,464 test records with no domain appearing in both partitions.

Model Development and Evaluation

Four models were fit against each feature set: a majority-class baseline, logistic regression on standardized inputs, a decision tree, and a random forest of 300 trees (Breiman, 2001), all implemented in scikit-learn (Pedregosa et al., 2011). Table 1 reports held-out performance on the stratified split.

Table 1

Held-Out Test Performance by Feature Set and Model, Stratified Split

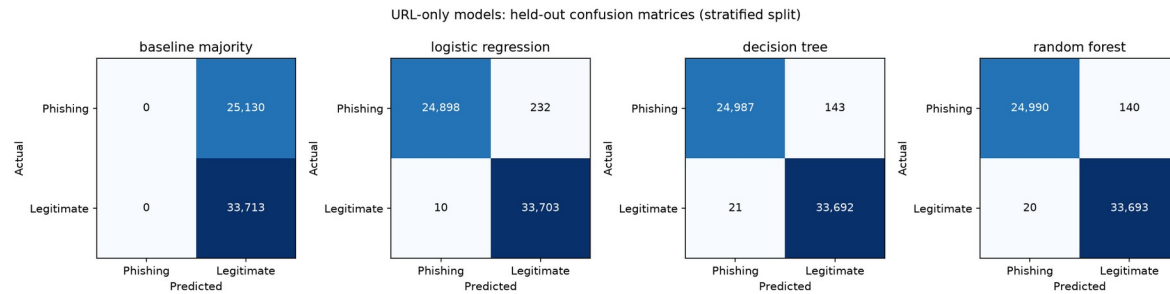
Feature set	Model	Accuracy	F1 (phishing)	ROC AUC	CV F1 (train)
full	Majority baseline	.5729	.0000	.5000	—
full	Logistic regression	.9999	.9999	1.0000	.9998
full	Decision tree	1.0000	1.0000	1.0000	1.0000
full	Random forest	1.0000	1.0000	1.0000	1.0000
no_derived	Majority baseline	.5729	.0000	.5000	—
no_derived	Logistic regression	.9993	.9991	1.0000	.9991
no_derived	Decision tree	.9990	.9988	.9990	.9987
no_derived	Random forest	.9999	.9998	1.0000	.9998
url_only	Majority baseline	.5729	.0000	.5000	—
url_only	Logistic regression	.9959	.9952	.9981	.9954
url_only	Decision tree	.9972	.9967	.9967	.9966
url_only	Random forest	.9973	.9968	.9981	.9968

Note. $n = 58,843$ test records. Phishing (label 0) is the positive class for precision, recall, and F1. CV F1 is the five-fold cross-validated mean on the training partition. Full metrics and per-model confusion matrices are recorded in `results/metrics_stratified_<feature_set>_<model>.json`.

The design anticipated a substantial decline across the ablation, which would have quantified how much of the benchmark result was artifact. The decline did not occur. Random forest accuracy moves from 1.0000 on full to .9999 on `no_derived` to .9973 on `url_only`. Removing 32 features, including the one that restates the label and all 26 page-content features, costs roughly a quarter of a percentage point. The flatness of Table 1 is the finding. Had the artifact been localized in the features that were removed, accuracy would have fallen when they were removed.

The most plausible competing explanation is that the lexical models memorize domains rather than learn URL structure. The domain-grouped partition tests that directly. Random forest accuracy on `url_only` was .9975 under the grouped partition, marginally higher than under stratified sampling, which rules memorization out. On the full feature set, the grouped decision tree and random forest both reached a cross-validated F1 of 1.000000 with a standard deviation of .000000 and a test F1 of .9999, while sharing no domain between training and test. Leakage that survives a domain split is not domain leakage.

Operationally, the `url_only` random forest reaches .9992 phishing precision and .9944 recall, missing 140 of 25,130 phishing URLs and raising 20 false alerts across 33,713 legitimate ones (Figure 4). The majority baseline, by contrast, achieves .5729 accuracy and a phishing F1 of .0000 in every configuration, because a classifier that always predicts the larger class never identifies a phishing URL. Those error counts are not a deployment estimate. They describe the same construction artifact viewed through a narrower window, and they carry no information about performance on live traffic.

Figure 4*Held-Out Confusion Matrices for the Four url_only Models*

Note. Stratified split, $n = 58,843$. The majority baseline identifies none of the 25,130 phishing URLs in the test partition. Each learned model closes nearly all of that gap using only lexical features available before a page is fetched.

Evaluation Metrics

No single metric is sufficient for a phishing classifier, and on this dataset accuracy is the least informative of the set, because artifact separation inflates it directly. Table 2 states the question each reported metric answers and the role it plays in this evaluation.

Table 2*Reported Metrics and Their Role in This Evaluation*

Metric	What it answers and why it is reported here
Accuracy	Share of all URLs classified correctly. Carried only for comparison to the published benchmark; class-construction artifacts inflate it directly, which makes it the least informative metric in this evaluation.
Precision	Share of phishing predictions that are correct. Quantifies false alerts, which cost blocked access and analyst attention.
Recall	Share of phishing URLs identified. Quantifies missed phishing, the primary security failure.
F1	Harmonic mean of the two. Reported for phishing rather than as a class average, so a high legitimate-class score cannot mask poor phishing detection.
ROC AUC	Threshold-independent class separation. Also inflated by artifacts, so it is read alongside the leakage findings rather than on its own.
Confusion matrix	Raw counts of each error type. Required for the false-negative and false-positive discussion, which summary statistics obscure.
Cross-validated F1	Mean F1 across five training folds. A stability check against a single held-out score; a standard deviation of .000000 on a perfect score is itself diagnostic.

Note. Precision, recall, and F1 are computed with phishing (label 0) as the positive class. Accuracy and ROC AUC are unaffected by that choice.

One convention requires explicit statement because it inverts the meaning of three of these metrics. Label 1 denotes a legitimate URL and label 0 denotes phishing. Scikit-learn treats the greater label as positive by default, so precision, recall, and F1 computed without specification describe detection of legitimate URLs under headings that read as phishing detection. The positive class is set to phishing once in the configuration module and threaded through every metric call and every stored metrics file. Any figure quoted from this project should carry that convention with it.

Ethical Considerations

Responsible Communication of Performance

The primary obligation is to report what the experiment establishes and nothing beyond it. Describing the classifier as 99.7% accurate, without qualification, implies that the same effectiveness should be expected against live phishing traffic. The evidence supports a narrower claim. The url_only random forest achieves .9973 accuracy on this project's frozen PhiUSIIL test partition, and the class-constancy findings in Section 2 give direct reason to expect that figure will not transfer. The NIST (2023) AI Risk Management Framework names validity and reliability, security and resilience, accountability, and transparency among the characteristics of trustworthy systems. Applied here, those principles require publishing the class convention, the feature availability at decision time, the leakage findings, and the evaluation procedure alongside any number the report quotes.

False Negatives and Missed Phishing

A phishing URL classified as legitimate is a security failure. Depending on deployment context, a missed site can expose a user to credential theft, financial fraud, or a malicious download. Phishing recall and the raw count of missed URLs therefore belong in the evaluation rather than buried inside overall accuracy. The url_only random forest misses 140 of 25,130

phishing URLs in the held-out partition, a recall of .9944. That is a strong within-dataset result, and the residual still represents 140 URLs that would reach a user.

False Positives and Operational Harm

Incorrectly flagging a legitimate URL blocks access to a valid resource, creates user friction, and contributes to alert fatigue at scale. The same model raises 20 false alerts across 33,713 legitimate URLs. The appropriate balance between missed phishing and false alerts depends on deployment context and risk tolerance, and it cannot be set from accuracy alone. Reporting both error types with their counts, rather than a single summary statistic, is what allows that judgment to be made by whoever bears the consequences.

Transparency and Reproducibility

A result that cannot be traced or reproduced cannot be audited. The project addresses this structurally rather than by assertion. One seed and one frozen split are shared across all members, analytical logic lives in a package rather than in notebook cells, and results are persisted as JSON and CSV before any section cites them. The dataset itself is not committed; the loader instead validates a SHA-256 fingerprint of the cleaned URL column, so five independently downloaded copies can be checked for identity. Every number in this report is read from a file in the results directory. The honest limit of that discipline is that it establishes reproducibility within this dataset only. External and temporally separated data would be required before any claim about production detection.

Team Collaboration Process

Group 5 organized around a shared GitHub repository with five defined roles and a written contract governing how numbers move between members. The contract holds that the seed and the train/test assignments are generated once and read by everyone, that no member loads the raw dataset directly, that analytical logic lives in the package rather than in notebook cells, and that every reported figure is written to disk before any section cites it. The last rule is the operative one, because a report assembled by five authors fails most often when two of them quote the same quantity from different runs. Work proceeded on per-role branches merged into

the trunk through pull requests, with day-to-day coordination in a group chat and code review at each merge.

Divya Kamepalli built the cleaning pipeline, the frozen stratified and grouped splits, the single-feature leakage screen, and the preprocessing narrative. Zubair Ahmed Baig Mogal wrote the exploratory analysis module and its figures, including the modal-share comparison that carries the collection-artifact finding and the single-feature leakage chart, and revised the module after review to read the data steward's results file rather than regenerate it. Alberto Kabore wrote the model driver with its archiving and provenance logging, the model roster and evaluation function, and the initial model development narrative. Bharadwaj Reddy Minukuri wrote the evaluation strategy, the metric justification, and the ethical considerations. J. Eric Collier wrote the project scaffold, the configuration module, the repository contract, the introduction and this section, and executed the final metrics run of September 20 on a refreshed environment using Kabore's driver.

Two process findings are recorded because each was caught by the review workflow rather than by the member who introduced it. The evaluation function initially used library defaults, which silently reported detection of legitimate URLs under labels reading as phishing detection; the modeling owner identified the inversion independently while validating output and derived the phishing metrics from the confusion matrix, after which the defect was corrected at its source. Separately, the repository README continued to assert a performance gap for six days after the measured results contradicted it, while its technical sections were revised normally. A partially current document is more dangerous than an obviously stale one, because the updated portions vouch for the rest. The rule adopted in response, and carried into Part 2, is that a claim about results either cites a file in the results directory or is marked as expected and unverified.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
<https://doi.org/10.1023/A:1010933404324>
- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), Article 100804.
<https://doi.org/10.1016/j.patter.2023.100804>
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), Article 15. <https://doi.org/10.1145/2382577.2382579>
- National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). U.S. Department of Commerce.
<https://doi.org/10.6028/NIST.AI.100-1>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Prasad, A., & Chandra, S. (2024). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. *Computers & Security*, 136, Article 103545. <https://doi.org/10.1016/j.cose.2023.103545>
- Sahingoz, O. K., Buber, E., Demir, O., & Diri, B. (2019). Machine learning based phishing detection from URLs. *Expert Systems with Applications*, 117, 345–357.
<https://doi.org/10.1016/j.eswa.2018.09.029>
- Sharda, R., Delen, D., & Turban, E. (2023). *Business intelligence, analytics, data science, and AI: A managerial perspective* (5th ed.). Pearson.